

SE3002 Assignment 02 Report

SE3002 Assignment 02 — Structural Testing of PX4- Autopilot v1.17.0

1. Group details and baseline

Student	Name	Roll number	Primary area
S1	Taimoor Shaukat	24i3015	environment, baseline, coverage, gaps
S2	Muhammad Bilal Tahir	24i3166	DataValidator + DataValidatorGroup, MC/DC
S3	Ali	24i3158	FailureDetector/Injector, findings, report

Section B, Course SE3002 Software Quality Engineering, Assignment 02.

Baseline: tag v1.17.0, built from commit d6f12ad1c4f70ad3230afd7d86e971421e02fef4 (this is the commit the tag points to via v1.17.0^{commit}, not the annotated tag object hash a5eb12d2ab591251faa009f76b2685b8cc64405d, which is a distinct git object — citing the tag object instead of the commit it resolves to is a documented mistake to avoid, see work/explain/P01.md). Work was carried out on branch sqe-a2, with merge-base with v1.17.0 confirmed equal to the required commit.

2. Local environment and setup evidence

All tooling versions were captured directly, not assumed: Ubuntu 24.04.4 LTS, x86_64, 13th Gen Intel Core i7-1355U (12 cores), 30 GiB RAM, kernel 7.0.0-34-generic. Compiler: GCC/G++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1); cmake 4.4.3; ninja 1.13.2; GNU Make 4.3; Python 3.12.3; lcov/genhtml 2.0-1; gcov 13.3.0; git 2.43.0. No Clang was installed or required — the macOS Coverage-build flag fix documented in the project's reference material does not apply to this Linux/GCC path.

PX4's own setup script (Tools/setup/ubuntu.sh) installs its toolchain via apt-get, which requires sudo; this environment had no non-interactive sudo available (sudo -n true failed). Rather than block on that, every missing tool was installed user-locally instead, with no elevation: cmake/ninja via pip install --user; lcov/genhtml 2.0 built from the upstream lcov source with make install PREFIX=~/.local; the one missing Perl module (DateTime) installed into a user-local local::lib/cpan environment. gcc/g++/gcov and python3 were already present system-wide. ~/.bashrc was updated to put ~/.local/bin on PATH. This is a legitimate, reproducible deviation from the documented sudo-based path, not a shortcut around a requirement; a reader with working sudo access can use the standard Tools/setup/ubuntu.sh script instead (Appendix A documents both).

Build/test commands used throughout: tools/sqe_run_tests.sh build|run|probes|shuffle and tools/sqe_coverage.sh build|baseline|student|final (wrappers around cmake --build build/px4_sitl_test and ctest), never upstream make tests_coverage (which runs make clean, omits branch data, and silently ignores lcov mismatches).

Baseline make tests (upstream suite, unmodified source, before any student test code existed): **100% tests passed, 147 of 147**, total test time 45.73s. No pre-existing failures were observed at baseline.

Second, independent environment. Because the primary machine needed a non-standard (no-sudo) toolchain, the whole submission was also reproduced on a second, standard environment built only with PX4's official Tools/setup/ubuntu.sh --no-nuttx --no-sim-tools: Windows 11 WSL2, Ubuntu 24.04.5 LTS, x86_64, GCC 13.3.0, cmake 3.28.3 (apt), ninja 1.11.1, lcov 2.0-1, from a fresh git clone --branch v1.17.0 --recursive (HEAD d6f12ad1, 0 uninitialised submodules). There the baseline again passed 147/147, the patch applied cleanly, all student tests passed, and the coverage hit sets were identical line-for-line and branch-for-branch to the primary machine's.

3. Repository analysis and scope selection record

Four areas (A-D) were selected after re-measuring candidates directly against the live v1.17.0 source (not copied from pre-derived reference material) via grep-based candidate screening and an upstream-gtest registration search (119 `px4_add_unit_gtest/px4_add_functional_gtest` lines, none matching `data_validator` or `failure_detector`).

Area A — `DataValidator.cpp` (`src/modules/sensors/data_validator/`): tracks one sensor instance's value stream and derives a 0.0–1.0 confidence from timeout, staleness and error-density/count, feeding the Area B voter. 155 lines, 58 executable lines, 15 decisions (DV-D01...D15), 0 compound decisions, 30 branch outcomes. No params, no uORB; all inputs are explicit `put()/confidence()` arguments. Test level: unit (`px4_add_unit_gtest`), since no `DEFINE_PARAMETERS/uORB` subscription/runtime-service construction occurs outside the diagnostic `print()` method.

Area B — `DataValidatorGroup.cpp` (MC/DC component): owns a linked list of `DataValidators`, ranks by confidence × priority, selects the “best” sensor and classifies failover events. 347 lines, 155 executable lines, 38 decisions (DVG-D01...D38), **5 compound decisions** (19 atomic conditions), 116 branch outcomes — the densest unit-level candidate. Confirmed call sites: `modules/sensors/voted_sensors_update.cpp` (IMU accel/gyro voting) and `modules/sensors/vehicle_magnetometer/VehicleMagnetometer.cpp` (magnetometer voting) — this logic directly decides which sensor instance feeds the state estimator. Test level: unit, same reasoning as A.

Area C — `FailureDetector.cpp`: detects attitude-envelope exceedance, ESC/telemetry faults, motor under-current, imbalanced-propeller vibration and external-ATS signals, aggregating into Commander's failsafe status bitmask. 354 lines, 153 executable lines, 44 decisions (FD-D01...D45; FD-D07 unused), **14 compound decisions**, 190 source-level branch outcomes — the largest and most logically dense area. 8 params read at construction (`FailureDetector.hpp:129-138`), 6 uORB subscriptions. Confirmed call site: `modules/commander/Commander.cpp:1857`. Test level: functional (`px4_add_functional_gtest`) — params are read once at construction, requiring the `set-params-then-construct` ordering; `gtest_functional_main` initializes uORB/params.

Area D — FailureInjector.cpp: reads `SYS_FAILURE_EN` and injected `vehicle_commands` to simulate motor failures for in-the-loop testing of Area C. 134 lines, 62 executable lines, 13 decisions (FI-D01... D13), 4 compound decisions, 48 source-level branch outcomes. Test level: functional, same reasoning as C.

Together A-D cover 990 source lines: **428 executable lines, 110 inventoried decisions, 23 compound decisions and 384 source-level branch outcomes** (exact: decisions from the per-file inventories, compound decisions counted from every `&&||` expression in the four files, lines and branches from `lcov`). (Earlier grep-based screening estimates in `work/scope/SCOPE_RECORD.md` — e.g. “7 compound” for Area B — were counting heuristics, superseded by these exact figures.) They form two safety chains (sensor redundancy A+B; failure detection C+D), both pure control/business logic with zero existing upstream GTest coverage.

Exclusions (file-spanning, D-004, flagged HUMAN-DECISION for team review): trivial header-only accessors (single-statement returns, no branches); the orphaned `src/modules/sensors/data_validator/tests/` directory (confirmed dead — `data_validator/CMakeLists.txt` never adds it, observation O-01); `print()` diagnostic-dump methods kept in scope for statement/branch coverage but excluded from MC/DC derivation as non-business-logic output.

Candidates rejected (re-measured against live source, none selected for the final scope): `lib/battery/battery.cpp` (stretch candidate, not needed); `land_detector/*` (heavy module-lifecycle harness, better suited to SITL); `HealthAndArmingChecks/checks/batteryCheck.cpp` (tightly coupled to shared check framework); `commander/failsafe/{failsafe,framework}.cpp` (upstream `failsafe_test.cpp` already exists, too large for the time box); `manual_control/ManualControlSelector.cpp` and `navigator/GeofenceBreachAvoidance` (already upstream-tested); `lib/hysteresis/hysteresis.cpp` (already upstream-tested, used only as a dependency of Area C); `commander/Safety.cpp` and `commander/UserModeIntention.cpp` (too small a decision surface). Re-verification corrected two REF_02 entries during candidate re-measurement: `Safety.cpp` and `UserModeIntention.cpp` were listed there as upstream-tested, but no file-specific gtest registration was found for either — the rejection verdict is unaffected, but the stated reason was corrected for accuracy.

4. Testing approach

Decisions were derived per-file from the live v1.17.0 source into per-area inventories (`work/basis/INVENTORY_DataValidator.md`, `INVENTORY_DataValidatorGroup.md`, `INVENTORY_FailureDetector_Injector.md`) plus three control-flow

sketches (CFG_DataValidator_confidence.md, CFG_DataValidatorGroup_get_best.md, CFG_FailureDetector_updateMotorStatus.md) before any test code was written. A setup map (work/basis/SETUP_MAP.md) records, per area, exactly how each decision is controlled: A/B take every input as an explicit function argument (put(), confidence(), get_best()); C/D require the ordering params-set → object-constructed → uORB-published → update()-called, since DEFINE_PARAMETERS is read once at construction and uORB has latest-sample (not queued) semantics for single-instance topics.

Oracles follow SPEC_02 §3's allowed sources: return values, getters, published uORB messages, flags/bitmasks and counters, derived from code comments/headers/documented library contracts or independently-derived arithmetic — never the production expression re-copied into the test (a circular oracle) and never “does not crash” alone. Several tests are explicitly **characterization tests** (asserting actual behaviour where the specification is silent/ambiguous, labelled characterization (F-nn) in their header comment per SPEC_02 §3): DV07 (all-NaN stream), DV19 (print()'s side effect), DV20 (timestamp-0 sentinel) and FD33 (motor-mask-survives-disarm). These are reported as characterizations, not specification-backed oracles, and are cross-referenced to their corresponding findings in §7.

Positive controls for “stays false” tests. A test that only asserts a flag stays False passes just as well if its setup never reached the decision at all (e.g. a uORB message that was never delivered). Every such test (SQE-FD-03/04/06/07/09/13/15/19/20/21) therefore ends with a positive control: the same detector (or an identical one built with the guarding parameter enabled) receives the triggering stimulus and must report True. This found a genuine test defect: SQE-FD-07 originally used a 170° pitch, which as Euler angles is pitch 10° / roll 180° (Euler pitch only spans ±90°), so the pitch flag could never trip even with the check enabled and the original assertion proved nothing. It now uses 80°.

Timing determinism. The test board has no lockstep clock (CONFIG_BOARD_NOLOCKSTEP=y), so FailureDetector's hysteresis and timeouts run on wall-clock time and the functional tests wait with sleep_for. Every wait is used only for a lower bound (“at least X ms have passed, so the flag must now be set”), which sleep_for guarantees regardless of machine load, because it never returns early. The only test that also needs an upper bound, SQE-FD-26 (the timer must be reset, so it must not have fired yet), was redesigned: a 1000 ms timeout, two 600 ms waits, both bounds measured from bracketing timestamps and asserted as explicit preconditions with 400 ms of slack, and a final positive control. A scheduler stall would therefore be reported as an environment problem, never as a wrong product result. The original version

ran a 10 ms window with no sleep, which would latch the mask permanently if the test thread was descheduled for more than 10 ms.

Independence and repeatability: every one of the 6 binaries was run with `--gtest_shuffle --gtest_repeat=10`, with zero failures: unit-SqeDataValidator 20 active tests, unit-SqeDataValidatorGroup 35, unit-SqeDataValidatorGroupAlloc 3, functional-SqeFailureDetector 36, functional-SqeFailureDetectorImu 8, functional-SqeFailureInjector 13 — 115 active tests in total, 10/10 shuffled repeats each. Each of the 115 tests also passes when run alone under `--gtest_filter`.

The full 1:1 test-to-decision mapping (120 inventory rows: 20 SQE-DV, 13 SQE-DVG, 3 SQE-DVG-AF, 22 SQE-DVG-MC, 36 SQE-FD, 8 SQE-FDI, 13 SQE-FI, 5 SQE-PRB probes) is not duplicated here — see `work/inventory/test_inventory.csv` and the “Test Inventory” sheet of the submitted workbook (`deliverables/24i3015_24i3166_24i3158_B.xlsx`).

5. MC/DC component selection and interpretation

DataValidatorGroup was chosen for the assignment’s one required MC/DC component because, among the four scoped areas, it combines the highest compound-decision density at the unit level (5 genuinely compound Boolean decisions, 2–7 atomic conditions each, pure functions of explicit arguments/internal state with no params/uORB/hidden I/O) with the clearest safety argument: its `get_best()` decides which physical sensor feeds the EKF state estimator, and `failover_index()/failover_state()` report when and why a failover occurred. FailureDetector has more compound decisions in absolute count but is covered to decision/branch level only, per the assignment’s “one justified component” requirement (D-003).

The 5 decisions: DVG-D13 (`get_best()` candidate-switch test, L187-190, 7 conditions A–G); DVG-D14 (“did best change”, L203, 3 conditions H/I/J); DVG-D15 (“real failsafe vs priority preference”, L207-208, 3 conditions K/L/M); DVG-D32/DVG-D34 (`failover_index()/failover_state()`, L282-283/L301-302, identical 3-condition pattern P/Q/R). 29 target-evaluation rows across 22 distinct test IDs implement these five decisions with **unique-cause independence pairs under short-circuit relaxation**: in every pair the target condition flips, the decision outcome flips, and every other condition that is evaluated in both tests holds the same value — but, because C++ `&&/||` short-circuit, a condition may be evaluated in one test of a pair and not evaluated at all in the other (recorded as NE(x) in the matrix). Example: in the A pair SQE-DVG-MC-07/08, C is not evaluated in MC07 (A&&B already

true) but is evaluated True in MC08. This is the standard short-circuit form of unique-cause MC/DC (no condition that is evaluated in both tests changes), not strict unique-cause with all other conditions identical, and not masking MC/DC (which would allow evaluated conditions to change).

Three worked examples. The **E pair** (DVG-D13, `fabsf(confidence-max_confidence)<0.01f`): SQE-DVG-MC-01 seeds sensor 0 at confidence 0.95/priority 50, then introduces sensor 1 at confidence 0.95/priority 75 ($|c-m|=0<0.01f$, $E=True$); its independence pair SQE-DVG-MC-03 changes only the candidate's confidence to 0.93 ($|0.93-0.95|=0.02f\geq0.01f$, $E=False$), holding every other condition's evaluated value fixed, and the decision outcome flips (switch vs no switch).

The **K pair** (DVG-D15, `pre_check_prio != -1`): $K=False$ happens only when `_curr_best` is -1 at entry — on the first `get_best()` call, or after a total failure has reset it to -1 (L235, L241). D15 is then False, so its body (L210-215) is skipped, and `_curr_best < 0` at L219 sends the call down the initial-bookkeeping branch, which never reads `true_failsafe` — so the $K=False$ row (SQE-DVG-MC-21) cannot be distinguished from its True counterpart (SQE-DVG-MC-01) by any API return value. This row is classified **O3** (structural evidence only): line 210 (`true_failsafe = false;`) shows 0 hits for MC21 versus 1 hit for MC01 in their respective per-test `.info` captures (line 207, where D15 is evaluated, is hit in both), reproduced independently on the second machine. This is not a weaker choice but the strongest evidence that can exist: D15's only other effect, `best->reset_state()` at L214, is always a no-op. Whenever D15 is True, `best` was chosen in the same call by D13, whose G condition means `best->confidence()` returned > 0 , and `DataValidator::confidence()` clears the error mask to `NO_ERROR` whenever it returns > 0 (`DataValidator.cpp` L134-138). So `reset_state()` writes the value the mask already holds, and no public getter can ever tell $D15=True$ from $D15=False$ in the K pair.

The **P pair** (DVG-D32/D34, shared by `failover_index()/failover_state()`): SQE-DVG-MC-13 and SQE-DVG-MC-25 form the P pair; SQE-DVG-MC-25 specifically constructs the “put with timestamp 0” scenario (`DataValidator.cpp`'s timestamp-0 sentinel, F-12) to drive the validator's `used()` state to False, which flips $P = next->used()$ while Q and R's evaluated values are held fixed.

Three condition combinations in DVG-D13 and one in DVG-D14 were determined **infeasible (with proof)**, not simply omitted: $B=T\wedge G=F$ and $F=T\wedge D=F$ are excluded because $B\Rightarrow G$ and $F\Rightarrow D$ are real implications of the underlying float/priority comparisons; $A=T\wedge B=T\wedge C=F$ is excluded because $(A\wedge B)\Rightarrow C$; and DVG-D14's $H=F\wedge I=F\wedge J=F$ is excluded by an algebraic trace showing $J=F$ forces $I=T$. A related float-precision characteristic (F-06: the 1% tie-break's $1.0f - d/100.0f$ is not exact at integer d , confirmed by

direct float32 arithmetic, e.g. $d=10 \rightarrow 0.89999998f$) is documented but deliberately not exercised by any of the 12 chosen D13 vectors, each kept ≥ 0.01 – 0.02 away from the boundary so the suite’s PASS/FAIL results are stable across runs.

The matrix was independently re-derived by hand and cross-checked with a standalone Python mirror of the exact C++ operator/short-circuit order; `tools/sqe_mcdc_check.py` reports ALL DECISIONS COMPLETE for all 5 decisions, confirming every condition has both a True-outcome and False-outcome evaluation with a traced unique-cause independence pair (short-circuit form, as defined above).

6. Coverage analysis (baseline vs final, raw vs source-level)

Baseline (upstream tests only, before any student test code): DataValidator.cpp 43/58 lines (74.1%), 21/30 branches (70.0%); DataValidatorGroup.cpp 101/155 lines (65.2%), 54/116 branches (46.6%); FailureDetector.cpp and FailureInjector.cpp 0/153 and 0/62 lines and 0/254 and 0/57 branches, since no upstream test exercises them. Scope total: 144/428 lines (33.6%), 75/457 raw branches (16.4%), 75/384 source-level branches (19.5%).

Final (full suite: 147 upstream tests plus the 6 student binaries, 153/153 passed):

File	Lines	Raw branches	Exception edges	Source-level branches
DataValidator.cpp	58/58 (100.0%)	30/30 (100.0%)	0	30/30 (100.0%)
DataValidatorGroup.cpp	155/155 (100.0%)	114/116 (98.3%)	0	114/116 (98.3%)
FailureDetector.cpp	153/153 (100.0%)	189/254 (74.4%)	64 (0 hit)	189/190 (99.5%)
FailureInjector.cpp	62/62 (100.0%)	47/57 (82.5%)	9 (0 hit)	47/48 (97.9%)
Scope total	428/428 (100.0%)	380/457 (83.2%)	73 (0 hit)	380/384 (99.0%)

A separate student-only capture (`ctest -R Sqe`, upstream suite excluded) gives identical numbers, so the student suite alone produces all of the scope’s coverage.

Which number measures what. `lcov`’s raw branch figure (83.2%) counts, besides the source decisions, one exception-unwind edge that GCC inserts after almost every function call

when C++ exceptions are enabled (lcof block IDs starting with e; gap G-05). None of the 73 is a decision in the source and none can be taken, because no call in the scope throws. The **source-level** figure (380/384, 99.0%) counts only the decision/condition outcomes in the source, classified by lcof's own exception marker; it is produced by tools/sqe_branch_split.py, which only reads the .info file. (lcof's built-in --rc no_exception_branch=1 was tried and, with lcof 2.0-1 and GCC 13, discards every branch record, so it could not be used.) In both figures, gcov's "branch" is an operand-level count for short-circuit &&|| (one branch pair per evaluated operand), not MC/DC; the MC/DC matrix in §5 is the separate, stronger claim for DataValidatorGroup.

Iterations. IT-1 (first full capture) investigated every uncovered item with raw gcov -b -c output and found 3 missing tests (FD-D25, FD-D38, FD-D41). IT-2 added SQE-FDI-08, SQE-FD-36 and SQE-FD-37 (FailureDetector 185 → 189 branches). IT-3 re-ran with no changes and reproduced IT-2 byte-for-byte. **IT-4 (post-audit revision)** closed two more gaps that had been classified as not coverable: G-01 (allocation failure — SQE-DVG-AF-01/02/03 in a separate binary with a NuttX-style operator new; lines 427 → 428) and G-04 (the print() " OFF" ternary, previously misdiagnosed as a gcov artefact — SQE-DVG-13). DataValidatorGroup.cpp went from 154/155 lines and 110/116 branches to 155/155 and 114/116.

Remaining gaps (4 source-level branches + G-05). Each has a location, class and written proof in work/GAPS.md; none was excluded to raise a percentage and no LCOV_EXCL_* marker or production change was made. - G-02 DataValidatorGroup.cpp:78 — the compiler-emitted null check inside delete (_first); the loop condition while (_first) guarantees the pointer is non-null, so the null side cannot occur. - G-03 DataValidatorGroup.cpp:213 best != nullptr False — infeasible: D15 True requires K True, and pre_check_prio leaves -1 only in the block that also assigns best (L162-168). - G-06 FailureInjector.cpp:43-44 param_get(...) == PX4_OK False (taken 65 times True, 0 False) — SYS_FAILURE_EN is a compiled-in parameter, so param_find() cannot return an invalid handle in this binary. - G-07 FailureDetector.cpp:194 copy() False directly after updated() True (7 True, 0 False) — impossible without a concurrent writer; the other copy() site, L222, is covered on both sides (6/1). - G-05 — the 73 exception edges above.

Annotated excerpts (from evidence/coverage/final_post_audit/html/.../DataValidatorGroup.cpp.gcov.html): at L187 (DVG-D13's first condition pair) branch 0 is taken 445 times and branch 1 30 times, and every branch pair on L187-190 has both outcomes taken, consistent with the MC/DC matrix's True/False coverage of all 7 conditions. At L213 branch 0 is taken 2 times and branch 1 (best == nullptr) 0 times while L210 (true_failsafe = false;) runs exactly 2 times, consistent with G-03. At L88 (!validator) both

outcomes are now taken (1 True from SQE-DVG-AF-01, 3 False) and L89 (return nullptr;) runs once; all 12 branch records on L260 (print())'s six flag ternaries) are taken.

7. Findings

All 15 candidate findings from the project's pre-derived register were independently re-investigated this session against the live source and the actual executed test/probe results; **14 had their code-level claim confirmed (F-13 only in part — half of it was refuted by test, see below), 0 rejected in full.** "Code-level claim confirmed" means the described behaviour really occurs; whether that behaviour is a defect is stated per finding below. Three findings (F-09, F-10, F-11) need a specification-level oracle judgment that the evidence alone cannot settle. The reasoning in work/FINDINGS.md argues F-09 and F-10 are defects and F-11 a specification ambiguity, but that classification was proposed by one team member and has not yet been agreed by the whole team, so F-09 and F-10 are reported below as **candidate defects (pending team review)**, not as confirmed defects.

Confirmed defects / observations (title — location — test ID — severity; full reproduction/expected/actual in work/findings/REPORT_BLOCKS.md, not duplicated here): - **F-01** best != nullptr guard (DataValidatorGroup.cpp:213) is dead code in its False branch — structural proof only, no severity (coverage-measurement artefact, cross-referenced to G-03 in §6/§8). - **F-02** get_best() returns a non-null stale-data pointer when *index == -1 signals total sensor failure, contradicting the header's "array of best values" phrasing — SQE-DVG-11, SQE-DVG-MC-19 confirm; SQE-PRB-03 (disabled probe) fails under the stricter nullptr oracle. Latent: both production callers discard the return value and use only *index. - **F-03** DataValidatorGroup(0) + add_new_validator() dereferences a null _last and crashes (confirmed via EXPECT_DEATH, SQE-DVG-10) — unreachable from production (all 3 real construction sites use {1}). - **F-04** the stale-value counter is shared across all 3 axes rather than per-axis, so a padded scalar sensor (e.g. airspeed, sensors.cpp:285) reaches "stale" roughly 3× faster than a true 3-axis signal — SQE-DV-05, documented design trade-off. - **F-05** DataValidator::print() is not read-only — its internal confidence() call can set ERROR_FLAG_TIMEOUT as a side effect of printing — SQE-DV-19 (characterization). - **F-06** the 1% confidence tie-break is float-rounding/optimisation-level-sensitive (confirmed by direct float32 arithmetic) — not exercised by the submitted suite's deliberately boundary-avoiding vectors; treated separately as an untested-interaction risk in §8. - **F-07a/F-07b (confirmed at runtime under sanitizers)** an unclamped ESC-count loop and an unguarded shift exponent in FailureInjector::manipulateEscStatus. In the normal/Coverage build the probes SQE-PRB-05/06 are inert SUCCEED() placeholders

and prove nothing; they were therefore run in sanitizer builds of only FailureInjector.cpp and the probe file (PX4's own AddressSanitizer/UndefinedBehaviorSanitizer build types do not compile on GCC 13 because of the bundled Abseil/fuzztest dependencies). F-07a: ASan reports stack-buffer-overflow, a READ at FailureInjector.cpp:118 (status_esc[8], one past the array) from SQE-PRB-05. F-07b: UBSan reports shift exponent 4294967195 is too large for 32-bit type 'int' at FailureInjector.cpp:120 from SQE-PRB-06. The 13 active FailureInjector tests run clean under both sanitizers. Reachable only through failure injection (SYS_FAILURE_EN=1), which is a test feature, so flight impact is limited to injection sessions. - **F-08** the FAILURE_TYPE_WRONG log message uses a 0-based motor index while every sibling case uses 1-based — cosmetic, confirmed by direct source read, no dedicated capturing test written (R4 bars manufacturing an assertion to pad the finding). - **F-09 (candidate defect, pending team review)** a stream of only non-finite samples reports confidence()==1.0/state()==NO_ERROR, contradicting the class's own stated purpose of identifying anomalies — SQE-DV-07 (characterization) records the as-shipped behaviour; SQE-PRB-01 encodes the proposed stricter oracle and fails against the as-shipped code. - **F-10 (candidate defect, pending team review)** _error_density == ERROR_DENSITY_WINDOW exactly yields confidence()==0 with no flag set (> not >= at DataValidator.cpp:125) — two outputs of the same object disagree at the same instant. SQE-DV-15 records the as-shipped behaviour; SQE-PRB-02 encodes the proposed stricter oracle and fails; consumer impact traced to voted_sensors_update.cpp:419. - **F-11 (specification ambiguity, not a defect)** an equal-priority, confidence-only sensor switch increments failover_count() even though the source's own comment frames the guard as distinguishing “a real failsafe” from “a priority preference” — SQE-DVG-MC-04, SQE-DVG-MC-07 confirm the current, self-consistent behaviour. Unlike F-09/F-10 this has no internal contradiction; resolving it requires knowing what the counter is used for downstream, which this session could not determine from source alone. - **F-12** timestamp 0 is an implicit “no data” sentinel in both put() and confidence() — SQE-DV-20, SQE-DVG-MC-25 confirm; not reachable in the real system since hrt_absolute_time() is never 0 after boot. - **F-14** (testability observation) two effects in get_best() cannot be observed through the public API: _first_failover_time has no getter, and best->reset_state() at L214 is always a no-op (the mask is already NO_ERROR, see §5). This is why DVG-D19's True/False outcomes and the K pair of DVG-D15 rely on structural (per-test coverage) evidence rather than a return value. - **F-15** FailureDetector's disarm-reset block clears the under-current mask but never the timed-out mask, so a previously-timed-out motor still reports failed in getMotorFailures() after a clean disarm — SQE-FD-33 (characterization).

Two citation corrections to the pre-derived register were made and documented rather than silently fixed: F-02's originally-cited evidence test ("MC18") is a stable no-failover control row with no bearing on the finding — the real evidence is MC19; F-12's originally-cited test ("MC17") does not exist in the actual suite — the real DVG-level evidence is MC25. Neither correction changes the finding itself.

F-13 (allocation failure; tested in the post-audit revision).

The register claimed both the constructor and `add_new_validator()` were unsafe when an allocation fails. A dedicated binary (`unit-SqeDataValidatorGroupAlloc`) now injects NuttX-style `nullptr` results from operator `new`. **Constructor: confirmed** —

`DataValidatorGroup(3)` whose 2nd allocation fails dereferences a null `prev` at L61 and crashes (SQE-DVG-AF-03); a single failed allocation in `DataValidatorGroup(1)` is safe (SQE-DVG-AF-02).

`add_new_validator()`: refuted — it returns `nullptr` and leaves the group intact (SQE-DVG-AF-01). Latent: only on a `nullptr`-returning allocator (NuttX), under memory exhaustion at start-up, and only for groups constructed with 3+ sensors; all production sites construct {1} and grow through the robust `add_new_validator()`.

Rejected candidates: half of F-13 (above) was refuted by executable evidence; no candidate was rejected in full. This is not taken as a sign of a confirmation-biased register: every retained finding has a reproducing test, characterization or sanitizer log that would have failed had the claim been false, and the F-13 test shows the process could and did reject a claim.

8. Gaps, limitations, residual risk, improvements

After IT-4 the structural gaps are 4 source-level branches (G-02, G-03, G-06, G-07) plus the 73 exception edges of G-05, each with a written proof in `work/GAPS.md` (§6 lists them). G-01 and G-04 were closed by new tests; G-04's original "tool artefact" classification turned out to be a misreading of `gcov`'s branch order and is corrected in `work/GAPS.md`.

Environment limits. The two machines used (§2) share one compiler family (GCC 13.3.0) and OS (Ubuntu 24.04); there was no cross-compiler build and no run on a NuttX target. The NuttX allocator behaviour behind G-01/F-13 was reproduced by a test-only operator `new` that returns `nullptr`, which exercises the same compiled `-fcheck-new` checks, but it is a simulation of the target allocator, not the target itself.

No SITL/HITL. This is a deliberate scope decision (§3, `work/scope/CANDIDATES.md` §4), not an oversight: none of the four scoped files touch drivers, the work-queue scheduler, module start/stop

lifecycle or simulator dynamics to reach their decisions, so unit/functional GTest levels reach every decision deterministically without SITL's added nondeterminism and wall-clock cost. The consequence is that this report says nothing about hardware timing, real sensor-driver interaction, or the actual multi-process/multi-thread behaviour of `voted_sensors_update.cpp/Commander.cpp` calling into the scoped files — those call sites are confirmed to exist (§3) but their real runtime interaction with the scoped logic was never executed.

F-06 as an untested-interaction risk. The 1%-confidence tie-break's float-rounding/optimisation-level sensitivity (§5, §7) is a specific, named risk that the coverage and MC/DC results in this report do not bound: the MC/DC matrix's 12 D13 vectors were deliberately kept away from this boundary for test stability, which means the suite provides no evidence about behaviour exactly at that boundary under a different optimisation level (e.g. a -02/-03 flight build) than the -00 Coverage build used throughout this session.

Concrete improvements (listed per SPEC_06 §8's guidance; (1) and (2) were carried out in the post-audit revision, the rest were not implemented): (1) **done:** a dedicated allocation-fault binary closed G-01 and gave F-13 runtime evidence (§6, §7); a further step would be the same tests on a real NuttX target; (2) **done:** sanitizer runs of the F-07 probes (§7), now with runtime ASan/UBSan reports; extending that to the whole suite would need PX4's sanitizer build types to be fixed for GCC 13, or a Clang toolchain; (3) a second optimisation-level build (-02) to directly measure F-06's adjacent-step divergence instead of relying on hand float32 arithmetic; (4) a SITL or lockstep-clock harness for FailureDetector, which would replace the wall-clock waits in the functional tests (§4) with simulated time; (5) the F-07a/F-07b fixes (clamp `esc_count` to `CONNECTED_ESC_MAX`, guard `i_esc` as FailureDetector already does), which are production changes and outside this assignment's scope.

9. Final quality judgment

The structural evidence in this report supports specific, bounded claims about the analysed scope — four files, `DataValidator.cpp`, `DataValidatorGroup.cpp`, `FailureDetector.cpp` and `FailureInjector.cpp` — and nothing broader. The student suite executes every one of the 428 executable lines and 380 of the 384 source-level branch outcomes (99.0%; lcov's raw figure is 83.2% because it also counts 73 compiler-generated exception edges that no call in the scope can take). Each of the 4 remaining branches has a written infeasibility proof, and two gaps first judged uncoverable were closed with new tests. On `DataValidatorGroup`'s five redundancy-selection decisions, MC/DC was achieved with unique-cause independence pairs under short-circuit relaxation,

every condition taking both values, checked by return values, getters or, for the one row with no observable effect, per-test structural evidence. Every result was reproduced on a second, independently built machine with identical hit sets, and every binary passed ten shuffled repeats.

This gives high confidence that every implemented decision outcome in the scope was exercised at least once with a checked oracle. It is not a claim that the requirements are complete or that the code is correct for all inputs. The suite found real defects: an out-of-bounds read and an undefined shift in FailureInjector, confirmed at runtime by ASan and UBSan, and a null dereference when a middle allocation fails in DataValidatorGroup's constructor. Two further findings, F-09 and F-10, depend on a specification judgment and are reported only as candidate defects pending team review.

What limits a broader conclusion: testing was at unit and functional GTest level, isolated from the real sensor and estimator pipeline; the scoped files' interaction with their real callers (VotedSensorsUpdate, Commander, VehicleMagnetometer) was read, not executed; FailureDetector's timing was tested on wall-clock time, not simulated time; the NuttX allocator was simulated, not run on a target; one compiler family (GCC 13.3.0) and one optimisation level (-O0) were used, so F-06's float-rounding sensitivity under -O2/-O3 is unmeasured. No claim is made, or should be inferred, about PX4 as a whole or about any file outside the analysed scope.

10. AI-assistance record

AI (Claude Code) was used materially across all phases: source verification (re-reading production code directly rather than trusting pre-derived reference material before every derivation), build/coverage troubleshooting, test implementation (115 active GTest cases + 5 disabled probes after the post-audit revision), MC/DC derivation and independent cross-checking, coverage-gap classification, findings investigation, workbook generation, and this report's writing.

Verification practice: every test traces to a decision ID checked by tools/sqe_trace_check.py (clean, 0 errors/warnings); every MC/DC pair was independently recomputed against a standalone Python mirror of C++ operator/short-circuit order; every number in this report was re-read from its evidence file this session, not carried from memory. No assertion was weakened to pass a test (R4). Test-design errors found and fixed: an early draft's wrong assumption about the ESC failure-mask bit layout (fixed before

any coverage capture used it), and, in the post-audit revision, SQE-FD-07's vacuous 170° pitch stimulus (exposed by the new positive control).

Key assumptions and how they were checked: every classification in `work/GAPS.md` and `work/FINDINGS.md` was checked by reading the cited file:line and, where needed, PX4/uORB internals. This was not infallible: the post-audit revision found that G-04 had been misclassified as a gcov artefact (the AI had read lcov's branch order left to right), and that F-13 was half wrong; both are corrected and documented. Two explicit, honestly-reported process assumptions: gate approvals (G00-G14) were self-approved autonomously under the user's standing session-start authorization, then later converted to a named human batch sign-off at session end (`work/STATUS.md`'s note states plainly this was a summary-level approval, not sequential per-gate review as intended); the three findings requiring a specification-level oracle decision (F-09, F-10, F-11) were resolved autonomously on one team member's authority, not full team deliberation — flagged for review before the viva, not presented as settled consensus.

Post-audit revision (also AI-assisted, Claude Code): an independent examiner-style audit rebuilt the submission on a second machine and confirmed every coverage number. It then led to these changes: F-07a/F-07b probes run under ASan/UBSan (now runtime-confirmed); exception-filtered branch coverage reported next to the raw figure; MC/DC wording corrected to “unique-cause under short-circuit relaxation”; F-09/F-10 downgraded to candidate defects pending team review; SQE-DVG-12 given real assertions in place of a bare `SUCCEED()`. A second pass added: the allocation-failure binary (closing G-01, runtime evidence for F-13), SQE-DVG-13 (closing G-04 after finding its original diagnosis wrong), positive controls for every “stays false” FailureDetector test (which exposed a vacuous stimulus in SQE-FD-07), a deterministic redesign of SQE-FD-26, exact decision counts in §3, and the K-pair unobservability proof in §5. Assumption introduced: the AI judged that the sanitizer reports are sufficient runtime evidence for F-07a/F-07b, and that the plain-build PASS of the two probes should not be reported. The team takes responsibility for all targets, expected results, tests and conclusions in this submission.

Appendix A — Reproduction commands

Clone and baseline (`work/explain/P01.md`):

```
git clone --branch v1.17.0 --recursive <px4-remote> PX4-Autopilot
cd PX4-Autopilot && git rev-parse 'v1.17.0^{commit}' # must
print d6f12ad1c4f70ad3230afd7d86e971421e02fef4
git checkout -b sqe-a2
```

```
git submodule status --recursive | grep -c '^-'          # must
print 0 (all submodules initialised)
```

Toolchain — standard path (requires working sudo):

```
Tools/setup/ubuntu.sh --no-nuttx --no-sim-tools
```

Toolchain — path actually used this session (no sudo available, D-002):

```
pip install --user cmake ninja
# build lcov/genhtml 2.0 from source: ./configure --
prefix=$HOME/.local && make install PREFIX=$HOME/.local
# install the DateTime Perl module via a user-local local::lib +
cpan, not system-wide
echo 'export PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc
```

Baseline build and test (P02):

```
make tests TESTFILTER=__no_tests__          # build only
make tests                                  # build + run (config
px4_sitl_test, ctest)
```

Coverage pipeline (P03-P10), via the project's own wrapper (never upstream make tests_coverage):

```
tools/sqe_coverage.sh build
tools/sqe_coverage.sh baseline              # pre-student-test
capture
make tests PX4_CMAKE_BUILD_TYPE=Coverage TESTFILTER=__no_tests__
# switch build dir to Coverage
tools/sqe_run_tests.sh build                # build all Sqe* test
binaries
tools/sqe_run_tests.sh run                  # ctest -R Sqe
tools/sqe_run_tests.sh shuffle              # --gtest_shuffle --
gtest_repeat=5|10 per binary
tools/sqe_run_tests.sh probes               # --
gtest_also_run_disabled_tests
tools/sqe_coverage.sh student <tag>         # student-suite-only
capture
tools/sqe_coverage.sh final <tag>           # full final capture
(HTML + .info)
```

Sanitizer runs of the F-07 probes (post-audit; run from the Coverage build directory after the build above — each script compiles only FailureInjector.cpp (+ the probe file for ASan) with the sanitizer into /tmp, links a separate test binary, and leaves the Coverage build untouched):

```
PX4_BUILD=PX4-Autopilot/build/px4_sitl_test bash tools/
sqe_asan_probes.sh    # PRB-05 -> ASan overflow, L118
```

```
PX4_BUILD=PX4-Autopilot/build/px4_sitl_test bash tools/
sqe_ubsan_probes.sh # PRB-06 -> UBSan shift, L120
```

Equivalent plain commands without the wrapper scripts (used for the independent re-run, evidence/repro_independent/):

```
make tests PX4_CMAKE_BUILD_TYPE=Coverage
# baseline: clean v1.17.0, 147 upstream tests
LC="--rc branch_coverage=1 --ignore-errors
mismatch,inconsistent,unused,negative,empty,source,gcov"
lcov $LC --capture --directory build/px4_sitl_test -o all.info
lcov $LC --extract all.info '*/data_validator/DataValidator.cpp'
'*/data_validator/DataValidatorGroup.cpp' \
'*/failure_detector/FailureDetector.cpp' '*/failure_detector/
FailureInjector.cpp' -o scope.info
git apply 24i3015_24i3166_24i3158_B.patch
make tests PX4_CMAKE_BUILD_TYPE=Coverage TESTFILTER=__no_tests__
# build student tests
lcov --zerocounters --directory build/px4_sitl_test
(cd build/px4_sitl_test && ctest -R Sqe)
# student-only run, then capture as above
```

MC/DC and traceability checks (P06, P12):

```
python3 tools/sqe_mcdc_check.py work/mcdc/mcdc_matrix.csv
python3 tools/sqe_branch_split.py evidence/coverage/
final_post_audit/scope.info # raw vs source-level branches
python3 tools/sqe_trace_check.py --px4 PX4-Autopilot --inventory
work/inventory/test_inventory.csv \
--mcdc work/mcdc/mcdc_matrix.csv
```

Workbook and report:

```
python3 tools/sqe_workbook.py # generates deliverables/
<BASE>.xlsx
python3 tools/sqe_word_count.py deliverables/report/REPORT.md
```

Appendix B — Evidence index

Path	What it is
evidence/env/environment.md	tool/OS/compiler versions
evidence/baseline/summary.txt	baseline make tests result (147/147)
work/scope/SCOPE_RECORD.md, work/scope/CANDIDATES.md	scope selection and rejected candidates
work/basis/SETUP_MAP.md, work/ basis/CFG_*.md, work/basis/ INVENTORY_*.md	test-basis derivation

Path	What it is
work/mcdc/MCDC_ANALYSIS.md, work/mcdc/mcdc_matrix.csv, work/ mcdc/mcdc_check_report.md	MC/DC derivation and checker confirmation
evidence/coverage/baseline/ per_file.md	baseline coverage
evidence/coverage/ final_post_audit/ (scope.info, summary.txt, branch_split.md, html/, student/)	final coverage after IT-4: 428/428 lines, 380/457 raw, 380/384 source-level branches
evidence/coverage/final/ per_file.md, evidence/coverage/ final/html/	earlier final capture (IT-3), kept for the iteration history
evidence/coverage/ compare_baseline_final.md	baseline vs final comparison
work/coverage_iterations.md	IT-0..IT-4 iteration history
work/GAPS.md	gap clusters G-01...G-07 with proofs (G-01 and G-04 closed in IT-4)
evidence/coverage/pertest/ MC21_03_EVIDENCE.md, MC21_evidence.info, MC01_evidence.info	O3 structural-coverage evidence for the K pair
work/findings/REPORT_BLOCKS.md, work/FINDINGS.md	findings register and report- ready text
evidence/tests/xml/*.xml	per-binary GTest XML results
evidence/tests/logs/ *_shuffle.log	shuffle/repeat stability logs
evidence/tests/probes/*	disabled-probe execution evidence
evidence/tests/sanitizer/*.log	ASan/UBSan runs of SQE- PRB-05/06 and of the 13 active FailureInjector tests
evidence/coverage/post_audit/	intermediate re-capture after the first SQE-DVG-12 change (427/428, 376/457)
evidence/tests/logs/ shuffle_post_audit/	10x shuffled-repeat logs and plain-run logs for all 6 binaries (post-audit)
evidence/tests/logs/ individual_post_audit.log	each of the 115 active tests run alone under --gtest_filter (115 pass)
tools/sqe_branch_split.py	raw vs source-level branch split of any .info file
evidence/repro_independent/	independent second-machine reproduction (baseline/

Path	What it is
	student/final .info, shuffle and probe logs)
work/inventory/ test_inventory.csv	full test-to-decision mapping (120 rows)
deliverables/ 24i3015_24i3166_24i3158_B.xlsx	testing workbook (2 sheets)
work/ai_assistance_log.md	full AI-assistance log
work/STATUS.md	phase/gate table, HUMAN- DECISION items
work/DECISIONS.md	ADR-lite decision log (D-002, D-004, D-008, D-009, D-010 cited above)